GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 3 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Binary trees and BSTs

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6	1
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Binary trees and BSTs | 7 questions | tap a title to jump to it

M	1. Expression Tree Single-Leaf Mutation	1Point3Acres	2026-09-27
M	2. Compute File System Total Size	1Point3Acres	2026-09-02
M	3. Nested List Weight Sum (LC 339) with Follow-Ups	1Point3Acres	2026-08-16
M	4. Longest Consecutive Path in Binary Tree	1Point3Acres	2026-07-02
M	5. Solve tree, graph, sliding-window problems	PracHub	2025-08-04
M	6. Streaming Insert + Find K-th Largest	1Point3Acres	2026-02-04
H	7. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	PracHub	2026-09-16

STUDY PATH: question 16 of 167 (Core 90)

Priority score 6.0 | Binary trees and BSTs | asked 23 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 1 of 7

Expression Tree Single-Leaf Mutation

ASKED 2026-09-27 | 23 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 203/228**

ABOUT THIS QUESTION

Topic: Core 90 - Binary trees and BSTs

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip. Current variants include XOR/OR/AND/NOT operators, no parent pointer on the node interface, and an expectation that the candidate cache parent links or paths during the initial traversal.SWEtreedfssimulationmediumstate-machineFrequencyLowLast asked2026-09-27Stageonsite-coding

APPROACH

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which operators exist (AND, OR, XOR, NOT), whether XOR and others are binary only, whether nodes have parent pointers, and whether leaves are flipped one at a time and then restored or cumulatively.

BRUTE FORCE TO SAY OUT LOUD

For each leaf flip it, evaluate the whole tree, and flip it back. O(n^2).

OPTIMAL IDEA

Evaluate once and cache each node's value. During that traversal record each leaf's path to the root (or parent links). To flip a leaf, recompute only the nodes on its path upward, stopping early when a parent's value does not change because the root value is then the same as before. That is O(depth) per flip.

DRY-RUN THESE CASES

A single leaf tree, a NOT at the root, XOR chains where every flip changes the root, OR with a true sibling (early stop).

LIKELY FOLLOW-UPS

Several flips applied together, or an undo.

INTERVIEW-DAY PLAN

PITFALLS

Mutating the cached values without restoring them between independent flips.

HOW TO THINK ABOUT IT

Flipping one leaf can only change the nodes on its path to the root, so cache every node's value and recompute only that path. The path is found through parent links you build during the first traversal. Stop early when a recomputed node keeps its old value, because nothing above it can change.

Basis: Written; incremental evaluation on a tree. Assumed rules (confirm them): leaves hold booleans; internal nodes are AND, OR, XOR (binary) or NOT (unary); report the root value after flipping each leaf on its own (flip, read the root, restore).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, leaves):
    answers = []
    for leaf in leaves:
        leaf.value = not leaf.value
        answers.append(evaluate(root))
        leaf.value = not leaf.value  # flip back
    return answers
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION · Cache values and parents, recompute only the path with early stop

TIME	SPACE
O(n + L * depth)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

FOLLOW-UP 2 Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

FOLLOW-UP 3 Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

FOLLOW-UP 4 Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Expression Tree Single-Leaf Mutation
#
# Evaluate a boolean expression tree, then answer the root
# value after each single-leaf flip. Current variants
# include XOR/OR/AND/NOT operators, no parent pointer on
# the node interface, and an expectation that the
# candidate cache parent links or paths during the initial
# traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The node interface has no parent pointer.
# 2. Apply several flips at once, or flips that stay.
# 3. Which leaves change the answer at all?
# 4. Which leaves can change the answer at all?
#
# INTERVIEW NOTES
# Flipping one leaf can only change the nodes on its path
# to the root, so cache every node's value and recompute
# only that path. The path is found through parent links
# you build during the first traversal. Stop early when a
# recomputed node keeps its old value, because nothing
# above it can change.
#
# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back
# For each of the L leaves, toggle it, evaluate the entire
# tree, and toggle it back. It is the correct baseline and
# the oracle for the cached version. Time O(n^2). Space
# O(h).


# HELPERS for the brute force
def _apply(op, a, b=None):
    return {
        "AND": lambda: a and b,
        "OR": lambda: a or b,
        "XOR": lambda: a != b,
        "NOT": lambda: not a,
    }[op]()


def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
        return not _eval_full(node.left)
    return _apply(
        node.op,
        _eval_full(node.left),
        _eval_full(node.right),
    )


def _leaves(root):
    out, stack = [], [root]
    while stack:
        n = stack.pop()
        if n.op is None:
            out.append(n)
        else:
            if n.right:
                stack.append(n.right)
            stack.append(n.left)
    return out


# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back | time O(n^2)
class XNode:
    def __init__(
        self, op=None, left=None, right=None, val=None
    ):
        self.op, self.left, self.right, self.val = (
            op,
            left,
            right,
            val,
        )


def flips_brute(root):
    out = []
    for leaf in _leaves(root):
        leaf.val = not leaf.val
        out.append(_eval_full(root))
        leaf.val = not leaf.val
    return out


# HELPERS for the base question
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a


def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None


def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
```

```
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves


# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
# parent.
def flip_answers(root):
```

```
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers


# FOLLOW-UP 3: Which leaves change the answer at all?
# Answer: Compute top-down 'sensitivity': a leaf matters
# if every ancestor's other input lets it through (AND
# with true, OR with false, any XOR or NOT). One pass
# gives all of them in O(n).
# FOLLOW-UP 4: Which leaves can change the answer at all?
# Answer: Propagate a 'matters' flag downward: through AND
# the other side must be true, through OR the other side
# must be false, through XOR and NOT it always passes.
def sensitive_leaves(root):
    # leaves whose flip can change the root value
    _, value, _ = build_cache(root)
    found, stack = [], [(root, True)]
    while stack:
        node, matters = stack.pop()
        if node.op is None:
            if matters:
                found.append(node)
            continue
        l, r = node.left, node.right
        # the flag passes if the other side lets it
        left_ok = right_ok = matters
        if node.op == "AND":
```

```
            left_ok = matters and value[id(r)]
            right_ok = matters and value[id(l)]
        elif node.op == "OR":
            left_ok = matters and not value[id(r)]
            right_ok = matters and not value[id(l)]
        stack.append((l, left_ok))
        if r is not None:
            stack.append((r, right_ok))
    return found


# HELPERS for the checks below
class Node:
    def __init__(self, op=None, left=None,
                 right=None, val=None):
        # op is AND, OR, XOR, NOT, or None for a leaf
        self.op, self.val = op, val
        self.left, self.right = left, right


def evaluate(node):
    # plain recursive evaluation (also the checker)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = evaluate(node.right) if node.right else None
    return combine(node.op, a, b)


if __name__ == "__main__":
    t, f = Node(val=True), Node(val=False)
    tree = Node("XOR", Node("AND", t, Node(val=True)),
                Node("NOT", f))
    leaves, answers = flip_answers(tree)
    # every cached answer matches a full re-evaluation
    for leaf, ans in zip(leaves, answers):
        leaf.val = not leaf.val
        assert evaluate(tree) == ans
        leaf.val = not leaf.val
    # the tree is unchanged afterwards
    assert evaluate(tree) is False
    # sensitivity agrees with really flipping each leaf
    base = evaluate(tree)
    for leaf in leaves:
        leaf.val = not leaf.val
```

```
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
        assert changes == (leaf in sensitive_leaves(tree))
```

NOTES FROM CANDIDATES

Brute force is: collect leaves by inorder traversal, flip each leaf, and re-evaluate the whole tree.
The natural optimization is to cache subtree values and only recompute the path from the flipped leaf to the root.
Clarify whether XOR is binary-only and whether the tree is guaranteed valid.
Early-stop optimization is expected: after a leaf flip, recompute upward only until a parent value stays unchanged, then reuse the previous root value.
Settle how intermediate update state will be represented before implementation. In an L4 onsite, uncertainty about that representation caused a promising approach to fall back to brute-force code and contributed to negative coding feedback.

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

STUDY PATH: question 17 of 167 (Core 90)

Priority score 5.0 | Binary trees and BSTs | asked 48 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 2 of 7

Compute File System Total Size

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 204/228**

ABOUT THIS QUESTION

Topic: Core 90 - Binary trees and BSTs

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

EXAMPLES

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

APPROACH

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the structure can contain cycles or shared directories (links), whether files have sizes only, and whether to handle very deep trees.

BRUTE FORCE TO SAY OUT LOUD

Recursive DFS adding file sizes. Simple and usually the expected first answer.

OPTIMAL IDEA

DFS with a visited set if cycles or shared nodes are possible, so each node is counted once. Use an explicit stack for deep trees. If queried repeatedly, memoize directory totals and invalidate ancestors when a file changes. O(n) per computation.

DRY-RUN THESE CASES

The example giving 65, an empty directory, a directory containing only directories, a symlink loop.

LIKELY FOLLOW-UPS

Cache totals, handle updates, or find the largest directory.

PITFALLS

INTERVIEW-DAY PLAN

Recursion depth on a long chain. Double counting shared nodes.

HOW TO THINK ABOUT IT

A total over a tree is a post-order sum. Two things change the answer in real file systems: depth (recursion limit) and sharing or cycles (the same node reachable twice must be counted once). A visited set solves both the cycle risk and the double counting, and an explicit stack solves the depth.

Basis: Written; a tree sum (like LeetCode 690 / 1448 style traversals). Assumed model: File(size) and Dir(children...); directories may be shared (links) in the harder version.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

TIME	SPACE
O(n)	O(depth) call stack

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute File System Total Size
#
# Examples: root = Dir( Dir(File(10), File(20)), File(5),
# Dir(File(30)), ) Total: 65
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is queried often and files change.
# 2. Hard links: the same file appears under two
# directories.
# 3. Find the largest directory.
#
# INTERVIEW NOTES
# A total over a tree is a post-order sum. Two things
# change the answer in real file systems: depth (recursion
# limit) and sharing or cycles (the same node reachable
# twice must be counted once). A visited set solves both
# the cycle risk and the double counting, and an explicit
# stack solves the depth.
#
# BRUTE FORCE (say it first): Plain recursion
# Sum a file's size, or the sum over a directory's
# children. Correct for ordinary trees and the answer most
# interviews want first. It overflows on very deep trees
# and loops forever on a cycle. Time O(n). Space O(depth)
# call stack.


import heapq


# HELPERS for the brute force
class File:
    def __init__(self, size):
        self.size, self.parent = size, None


# BRUTE FORCE (say it first): Plain recursion | time O(n)
def total_size_recursive(node):
    if isinstance(node, File):
        return node.size
    return sum(
        total_size_recursive(c) for c in node.children
    )
```

```
# HELPERS for follow-up 3
class Dir:
    def __init__(self, *children):
        self.children, self.parent = list(children), None
        # cached total; None means "needs recomputing"
        self.cached = None
        for child in children:
            child.parent = self


def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # post-order: a directory after all its children
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            stack.extend(node.children)
    for node in reversed(order):
        if isinstance(node, Dir) and node.cached is None:
            kids = node.children
            node.cached = sum(size_of(c) for c in kids)
    return root.cached


# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
# FOLLOW-UP 2: Hard links: the same file appears under two
# directories.
# Answer: Count a file once per identity (a visited set of
# ids, as coded) if the question asks for disk usage, but
# once per path if it asks for the sum of the sizes shown.
# Ask which.
def total_size(root, count_once=True):
```

```
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total


# FOLLOW-UP 1: The tree is queried often and files change.
# Answer: Cache each directory's total and, when a file
# changes, clear the cache only for its ancestors (walk
# parent links). A query recomputes only cleared nodes.
# Updates cost O(depth).
def resize(file, size):
    # a file changed: only its ancestors go stale
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def directory_totals(root):
    # the size of every directory, in one pass
```

```
    cached_total(root)
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            out.append(node.cached)
```

```
            stack.extend(node.children)
    return out


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def top_k_directories(root, k):
    # keep a heap of size k: O(n log k)
    return heapq.nlargest(k, directory_totals(root))


if __name__ == "__main__":
    inner = Dir(File(10), File(20))
    root = Dir(inner, File(5), Dir(File(30)))
    assert total_size(root) == 65
    assert cached_total(root) == 65
    # change one file: only its ancestors are recomputed
    first = inner.children[0]
    resize(first, 15)
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70
    assert top_k_directories(root, 2) == [70, 35]
    # a hard link: the same file under two directories
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14
```

NOTES FROM CANDIDATES

If the interviewer says "strict tree", drop the visited set and stick with plain DFS — extra cycle handling reads as over-engineering.
For the cache follow-up, store the cached total on each directory node; when a file changes, walk parent pointers invalidating along the path.
The interviewer also pushes for an iterative DFS (explicit stack) after the recursive solution lands — have both ready.

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

STUDY PATH: question 18 of 167 (Core 90)

Priority score 4.16 | Binary trees and BSTs | asked 65 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 3 of 7

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 16.5** | **30 DAYS 0** | **PRIORITY 156/228**

ABOUT THIS QUESTION

Topic: Core 90 - Binary trees and BSTs

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

EXAMPLES

```
"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2
    + 2×2 + 91×3 = 320.
Use [2, [91]] as a manual trace for bracket depth and
    the final-number flush.
```

APPROACH

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the input is already parsed or a string, whether negatives and multi-digit numbers appear, and whether weight is depth from the top (1 at the outside) or the inverse weight variant.

BRUTE FORCE TO SAY OUT LOUD

Parse into a tree of lists and recurse with depth. Simple but recursion can overflow.

OPTIMAL IDEA

For a string, scan once with a depth counter: '[' adds one, ']' subtracts one, and an integer is multiplied by the current depth when its last digit ends (flush the number on any non-digit and at the end of the string). For parsed lists use BFS level by level. O(n), O(1) extra for the scanner.

DRY-RUN THESE CASES

[3, 8, [2, 14], [2, [91]]] gives 320. A number glued to the closing bracket. A negative number. A deeply nested list.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

The inverse weight version (LeetCode 364), depth limit, or iterative versus recursive.

PITFALLS

Not flushing a trailing number. Treating '-' as a separate token.

HOW TO THINK ABOUT IT

The weight of a number is just the nesting depth where it appears, and the nesting depth is the number of '[' still open. That means you never need to build the nested structure: scan once with a counter, and flush a number whenever a non-digit arrives (and at the end).

Basis: LeetCode 339, Nested List Weight Sum, with the nested list given as a string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Nested List Weight Sum (LC 339) with
# Follow-Ups
#
# Examples: "[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×
2
# + 14×2 + 2×2 + 91×3 = 320. Use [2, [91]] as a manual
# trace for bracket depth and the final-number flush.
#
# FOLLOW-UPS (answered below, in this order):
# 1. When does the recursive (DFS) version become unsafe?
# 2. What is the inverse version (deeper integers weigh
# less)?
# 3. The inverse version (shallow integers weigh less,
# LeetCode 364).
#
# INTERVIEW NOTES
# The weight of a number is just the nesting depth where
# it appears, and the nesting depth is the number of '['
# still open. That means you never need to build the
# nested structure: scan once with a counter, and flush a
# number whenever a non-digit arrives (and at the end).
#
# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth
# json.loads turns the text into nested lists; a recursive
# function adds each integer times its depth. Clear, but
# it builds the whole structure and recurses as deep as
# the nesting. Time O(n). Space O(depth).
```

```
import json


# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth | time O(n)
def depth_sum_parsed(s):
    def go(items, depth):
        return sum(
            (
                x * depth
                if isinstance(x, int)
                else go(x, depth + 1)
            )
            for x in items
        )
```

```
    return go(json.loads(s), 1)


# HELPERS for the base question
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
```

```
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

```python
# FOLLOW-UP 2: What is the inverse version (deeper
# integers weigh less)?
# Answer: That is LeetCode 364. Do a level-by-level pass
# that keeps a running sum of everything seen so far and
# add it to the total once per level: total += running_sum
# on each level.
# FOLLOW-UP 3: The inverse version (shallow integers weigh
# less, LeetCode 364).
# Answer: Process level by level and keep a running sum of
# every integer seen so far; add the running sum to the
# total once per level that holds an integer or lies above
# one (trim trailing levels that contain only empty
# lists).
def inverse_weight_sum(nested):
    # deeper integers weigh less (LeetCode 364)
    pairs = list(with_depths(nested))
    if not pairs:
        return 0
    deepest = max(d for _, d in pairs)
    # weight = deepest + 1 - depth
    return sum(v * (deepest + 1 - d) for v, d in pairs)


if __name__ == "__main__":
    data = [3, 8, [2, 14], [2, [91]]]
    # 3 + 8 + 2*2 + 14*2 + 2*2 + 91*3 = 320
    assert weight_sum(data) == 320
    text = "[3, 8, [2, 14], [2, [91]]]"
    assert weight_sum_string(text) == 320
    assert weight_sum_string("[2, [91]]") == 2 + 91 * 2
    # inverse weights: [[1,1],2,[1,1]] -> 8
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    # empty lists add nothing
```

```python
    assert weight_sum([[], [[]]]) == 0
    # 20000 levels deep is still fine
    deep = []
    for _ in range(20000):
        deep = [deep, 1]
    assert weight_sum(deep) > 0
```

NOTES FROM CANDIDATES

The easy-to-miss bug is leaving the last buffered number uncommitted when the string ends. Manually tracing the small nested case exposed this during the interview, and fixing it proactively was part of the signal.
DFS visits each nested value once in O(n) time and uses O(d) call-stack space at maximum depth d; BFS is also O(n) time with a frontier proportional to the widest level. In the string parser, track the sign and whether a number is currently buffered, flush only when that flag is set, and flush before decrementing depth on].

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

STUDY PATH: question 19 of 167 (Core 90)

Priority score 3.16 | Binary trees and BSTs | asked 110 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 4 of 7

Longest Consecutive Path in Binary Tree

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 157/228

ABOUT THIS QUESTION

Topic: Core 90 - Binary trees and BSTs

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #549 Binary Tree Longest Consecutive Sequence II

LeetCode match: LC 549 Binary Tree Longest Consecutive Sequence II (related)

Frequency: Single report | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-longest-consecutive-path

QUESTION

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1. Follow-up: allow the path to extend in any direction through the tree, not only from parent to child.SWEtreedfsrecursionmediumFrequencySingle reportLast asked2026-07-02Stagephone-screen

APPROACH

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether length counts nodes or edges, whether the path goes only parent to child (base) or can bend through a node (follow-up), and whether the sequence must increase by exactly one.

BRUTE FORCE TO SAY OUT LOUD

From every node walk down both children recording the longest run. O(n^2) on a skewed tree.

OPTIMAL IDEA

Base: DFS passing the current run length; it grows by one when the child is exactly the parent plus one, otherwise it resets to 1; keep the global best. O(n). Follow-up: each node returns the longest increasing and longest decreasing chain going down from it; the best path through a node is inc + dec - 1.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A single node, an empty tree, duplicate values, a split branch, a descending chain.

LIKELY FOLLOW-UPS

The any-direction version (above), and an iterative version for very deep trees.

PITFALLS

Counting edges when the interviewer expects nodes. Resetting the run to 0 instead of 1.

HOW TO THINK ABOUT IT

Each node needs to know the longest run that ends at its parent, so pass the current length down and reset it to 1 when the value is not parent + 1. For the bending version each node must report two lengths upward (increasing and decreasing chains going down), and the best path through a node combines one of each.

Basis: LeetCode 298 (parent-to-child) and LeetCode 549 (any direction), Binary Tree Longest Consecutive Sequence I and II.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def walk(node):
    best = 1
    for child in (node.left, node.right):
        if child and child.val == node.val + 1:
            best = max(best, 1 + walk(child))
    return best
answer = 0
for node in all_nodes(root):
    answer = max(answer, walk(node))
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A node deep in a chain is walked again from every ancestor, O(n * h). Passing the current run length down in one traversal makes it O(n).

OPTIMAL SOLUTION · Top-down DFS carrying the current run

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

FOLLOW-UP 2 Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

FOLLOW-UP 3 Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

FOLLOW-UP 4 The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

FOLLOW-UP 5 The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Consecutive Path in Binary Tree
#
# Given a binary tree, return the length of the longest
# parent-to-child path whose values increase consecutively
# by 1. Follow-up: allow the path to extend in any
# direction through the tree, not only from parent to
# child.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does the any-direction version return for a
# single node, or for duplicate values?
# 2. Can you avoid recursion?
# 3. Return the path itself, not only its length.
# 4. The path may go in any direction (LeetCode 549).
# 5. The values must increase by 1 or decrease by 1 along
# the path but not mix.
#
# INTERVIEW NOTES
# Each node needs to know the longest run that ends at its
# parent, so pass the current length down and reset it to
# 1 when the value is not parent + 1. For the bending
# version each node must report two lengths upward
# (increasing and decreasing chains going down), and the
# best path through a node combines one of each.
#
# BRUTE FORCE (say it first): Start a downward walk from
# every node
# For each node, walk down both children while the next
# value is exactly one more, tracking the longest run.
# Correct and the oracle for the base question; nodes deep
# in a chain are re-walked from every ancestor. Time O(n *
# h). Space O(h).


# BRUTE FORCE (say it first): Start a downward walk from
# every node | time O(n * h)
def longest_consecutive_brute(root):
    best = 0

    def down(node):
        a = b = 0
        if node.left and node.left.val == node.val + 1:
            a = down(node.left)
        if (
            node.right
            and node.right.val == node.val + 1
        ):
            b = down(node.right)
        return 1 + max(a, b)

    def visit(node):
        nonlocal best
        if node:
            best = max(best, down(node))
            visit(node.left)
            visit(node.right)

    visit(root)
    return best


# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
def longest_consecutive(root, want_path=False):
    # parent -> child path whose values go up by exactly 1
    best, best_path = 0, []
    stack = [(root, None, [])] if root else []
    while stack:
        node, parent, run = stack.pop()
        # extend the run or start a new one at this node
        if parent is not None and node.val == parent + 1:
            run = run + [node.val]
        else:
            run = [node.val]
        if len(run) > best:
            best, best_path = len(run), run
        for child in (node.left, node.right):
            if child is not None:
                stack.append((child, node.val, run))
    # FOLLOW-UP 3: want_path=True returns the path itself
    return best_path if want_path else best


# FOLLOW-UP 4: The path may go in any direction (LeetCode
```

```
# 549).
# Answer: Return the pair (longest increasing chain down,
```

```
# longest decreasing chain down) from each node, and
# update the global best with inc + dec - 1 at the node.
# Verified against exhaustive path enumeration for small
# trees.
def longest_any_direction(root):
    # LeetCode 549: the path may bend at one node
    if root is None:
        return 0
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                stack.append(child)
    # inc[x] / dec[x]: longest chain going down from x
    inc, dec, best = {}, {}, 1
    # children are processed before their parents
    for node in reversed(order):
        up = down = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                up = max(up, inc[child] + 1)
            if child.val == node.val - 1:
                down = max(down, dec[child] + 1)
        inc[node], dec[node] = up, down
        # the bend joins both chains (counted node once)
        best = max(best, up + down - 1)
    return best


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


if __name__ == "__main__":
    #     1
    #    / \
    #   2   0
    #  /
    # 3
```

```
    left = TreeNode(2, TreeNode(3))
    tree = TreeNode(1, left, TreeNode(0))
    assert longest_consecutive(tree) == 3
    assert longest_consecutive(tree, True) == [1, 2, 3]
    # bending: 2 - 1 - 0 or 3 - 2 - 1 - 0 is length 4
    assert longest_any_direction(tree) == 4
    assert longest_consecutive(None) == 0
    assert longest_any_direction(None) == 0
    # duplicates never extend a chain
    dup = TreeNode(5, TreeNode(5), TreeNode(5))
    assert longest_consecutive(dup) == 1
    assert longest_any_direction(dup) == 1
    # a long chain needs no recursion
    chain = TreeNode(0)
    cur = chain
    for i in range(1, 5000):
        cur.left = TreeNode(i)
        cur = cur.left
    assert longest_consecutive(chain) == 5000
```

NOTES FROM CANDIDATES

In the base version, carry the current consecutive length while traversing downward and reset to 1 whenever the child is not exactly parent + 1.
For the any-direction follow-up, track increasing and decreasing chain lengths at each node, then combine through the node when the left/right child values support a consecutive bridge.
Clarify whether the answer is measured in nodes or edges before coding; Google candidates usually state node count.

PREPARATION

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

STUDY PATH: question 20 of 167 (Core 90)

Priority score 3.08 | Binary trees and BSTs | asked 442 days ago

MEDIUM | PracHub | Technical Screen | 5 of 7

★ TOP 75: rank 46 in the Medium and Hard priority list

★ Solve tree, graph, sliding-window problems

POSTED 2025-08-04 | 442 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 29/228

ABOUT THIS QUESTION

Topic: Core 90 - Binary trees and BSTs

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #103 Binary Tree Zigzag Level Order Traversal, LeetCode #207 Course Schedule

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal + LC 207 Course Schedule

Posted on PracHub: 2025-08-04 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/solve-tree-graph-sliding-window-problems

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters. https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/ https://leetcode.com/problems/course-schedule/description/ https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

QUESTION

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

Examples

Input: ("eceba", 2)

Expected Output: 3

Explanation: The longest valid substring is 'ece', which has length 3.

Input: ("aa", 1)

Expected Output: 2

Explanation: The whole string contains only one distinct character.

Hints

A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

APPROACH

BFS by level with reversed alternate levels; topological sort / DFS cycle detection; plus a sliding-window task.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Three LeetCode problems: 103 zigzag level order, 207 course schedule, 340 longest substring with at most K distinct characters. For each ask the usual edge questions: empty tree, duplicate prerequisites, K = 0.

BRUTE FORCE TO SAY OUT LOUD

Zigzag: do a normal level order and reverse alternate levels afterwards. Course schedule: DFS from every course to find cycles. Substring: check every substring.

OPTIMAL IDEA

Zigzag: BFS level by level, reverse (or use a deque appended on the left or right) on every other level. O(n). Course schedule: Kahn's algorithm, with in-degree counts and a queue; if processed courses equal the course count there is no cycle. O(V + E). Substring (340): sliding window with a dict of counts; expand right, and while distinct characters exceed K shrink from the left, deleting a character at count 0. Track the maximum window. O(n).

DRY-RUN THESE CASES

Zigzag: empty tree, a single node, a skewed tree. Course schedule: no prerequisites, a self loop, a two-course cycle. Substring: K = 0 (answer 0), K larger than distinct characters, a repeated character.

LIKELY FOLLOW-UPS

Course Schedule II (return an order), longest substring with exactly K distinct, at most K distinct in a stream.

PITFALLS

Reversing the wrong levels. Not removing a key from the window dict when its count hits zero.

HOW TO THINK ABOUT IT

Three standard patterns. Level order needs a queue and a direction flag per level. 'Can everything be finished' is cycle detection in a directed graph: Kahn's algorithm removes nodes with no remaining prerequisites, and a cycle leaves some behind. 'Longest window with at most K of something' is a sliding window with a count map that shrinks from the left when the limit is exceeded.

Basis: Three LeetCode problems: 103 Binary Tree Zigzag Level Order, 207 Course Schedule and 340 Longest Substring with At Most K Distinct Characters.

BRUTE FORCE · Course schedule: DFS from every course looking for a cycle

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Each search can walk the whole graph again, O(V * (V + E)). Kahn's algorithm (remove courses with no unmet prerequisites) is O(V + E).

BRUTE FORCE · Longest window: check every substring

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, K):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) > K:
                break
            best = max(best, j - i + 1)
    return best
```

TIME	SPACE
O(n^2)	O(k)

WHY IT IS SLOW

O(n^2) in the worst case. A sliding window with a count map that shrinks from the left when it exceeds K distinct characters is O(n).

BRUTE FORCE · Zigzag: level order, then reverse the odd levels

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time with one extra copy of every second level; filling each level from alternating ends of a deque avoids it.

OPTIMAL SOLUTION · Course schedule: Kahn's topological sort

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Longest window: sliding window with a count map

TIME	SPACE
O(n)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Zigzag: fill each level from the correct end

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Course Schedule II: return an order in which to take the courses.

The same Kahn loop records the order in which courses are removed; if fewer than n courses are removed, return an empty list.

FOLLOW-UP 2 Longest substring with exactly K distinct characters.

Take atMost(K) minus atMost(K - 1) for counting substrings, or keep the window and only record lengths when the distinct count is exactly K (careful with the shrink).

FOLLOW-UP 3 Zigzag on an n-ary tree, or the right-to-left variant.

Replace the two children by the child list, and swap which levels are appended on the left.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve tree, graph, sliding-window
# problems
#
# ##### Question LeetCode 103. Binary Tree Zigzag Level
# Order Traversal – given a binary tree root, return its
# zigzag level-order traversal. LeetCode 207. Course
# Schedule – given course prerequisites, determine if all
# courses can be finished by detecting cycles with
# topological sort. LeetCode 340. Longest Substring with
# At Most K Distinct Characters – given string s and
# integer k, return the length of the longest substring
# containing at most k distinct characters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Course Schedule II: return an order in which to take
# the courses.
# 2. Longest substring with exactly K distinct characters.
# 3. Zigzag on an n-ary tree, or the right-to-left
# variant.
#
# INTERVIEW NOTES
# Three standard patterns. Level order needs a queue and a
# direction flag per level. 'Can everything be finished'
# is cycle detection in a directed graph: Kahn's algorithm
# removes nodes with no remaining prerequisites, and a
# cycle leaves some behind. 'Longest window with at most K
# of something' is a sliding window with a count map that
# shrinks from the left when the limit is exceeded.
#
# BRUTE FORCE (say it first): Course schedule: DFS from
# every course looking for a cycle
# From each course follow prerequisites; if the walk
# returns to the start there is a cycle. Correct and the
# oracle; the same parts of the graph are walked again and
# again. Time O(V * (V + E)). Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Course schedule: DFS from
# every course looking for a cycle | time O(V * (V + E))
def can_finish_dfs(n, prereqs):
    adj = defaultdict(list)
    for a, b in prereqs:
        adj[b].append(a)

    def reaches(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(reaches(c) for c in range(n))


# HELPERS for the base question
# FOLLOW-UP 1: Course Schedule II: return an order in
# which to take the courses.
# Answer: The same Kahn loop records the order in which
# courses are removed; if fewer than n courses are
# removed, return an empty list.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def course_order(n, prereqs):
    # (a, b): to take a you must take b first. Kahn's
    # algorithm;
    # if some course never frees up there is a cycle
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
```

```
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []


# HELPERS for follow-up 2
```

```
def count_at_most_k(s, k):
    # number of substrings with at most k distinct letters
    if k < 1:
        return 0
    count, left, total = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        total += right - left + 1
    return total


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def zigzag(root):
    # level order, alternating direction
    out, left_to_right = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            # append for left-to-right, appendleft
            # otherwise
            if left_to_right:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        left_to_right = not left_to_right
    return out


# BASE QUESTION (optimal): Course schedule: Kahn's
```

```
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def can_finish(n, prereqs):
    return len(course_order(n, prereqs)) == n


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
```

```
# of the courses that depend on it, and queue those that
# reach zero.
def longest_at_most_k(s, k):
    # sliding window with a count of each letter inside it
    count, left, best = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        # too many distinct letters: shrink from the left
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


# FOLLOW-UP 2: Longest substring with exactly K distinct
# characters.
# Answer: Take atMost(K) minus atMost(K - 1) for counting
# substrings, or keep the window and only record lengths
# when the distinct count is exactly K (careful with the
# shrink).
def count_exactly_k(s, k):
    # exactly k = at most k minus at most k - 1
    most = count_at_most_k(s, k)
    return most - count_at_most_k(s, k - 1)


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val, self.left, self.right = val, left, right


def build(values):
    # level-order list, None for a missing child
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


if __name__ == "__main__":
    tree = build([3, 9, 20, None, None, 15, 7])
    assert zigzag(tree) == [[3], [20, 9], [15, 7]]
    assert zigzag(None) == []
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    order = course_order(4, [(1, 0), (2, 1), (3, 2)])
    assert order == [0, 1, 2, 3]
    assert course_order(2, [(0, 1), (1, 0)]) == []
    assert longest_at_most_k("eceba", 2) == 3
    assert longest_at_most_k("aa", 1) == 2
    assert longest_at_most_k("abc", 0) == 0
    assert count_exactly_k("abab", 2) == 6
    assert count_at_most_k("abab", 1) == 4
```

FUNCTION AND TEST CASES

Signature

```
def solution(root: list[str]) -> list[list[int]]:
```

Sample call

```
solution([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
([3, 9, 20, None, None, 15, 7],)	[[3], [20, 9], [15, 7]]
([1],)	[[1]]

STUDY PATH: question 21 of 167 (Core 90)

Priority score 2.8 | Binary trees and BSTs | asked 258 days ago | frequency Single report

MEDIUM | IN APP LC 703 | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 6 of 7

★ TOP 75: rank 68 in the Medium and Hard priority list

★ Streaming Insert + Find K-th Largest

ASKED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 118/228

ABOUT THIS QUESTION

Topic: Core 90 - Binary trees and BSTs

Optimal answer from: your app's interview approach (LeetCode #703)

LeetCode match: LC 703 Kth Largest Element in a Stream

Frequency: Single report | Last asked: 2026-02-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/insert-find-kth-largest

QUESTION

EXAMPLES

```
insert(3)
insert(3)
insert(2)
findLargest(0) -> 3
findLargest(1) -> 3
findLargest(2) -> 2
```

APPROACH

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

HOW TO THINK ABOUT IT

If k can change from query to query you need every value, in a structure that supports rank queries; if k is fixed you only need the k largest, and a min-heap of size k gives the answer at its root. Ask which of the two it is: it decides everything.

Basis: LeetCode 703, Kth Largest Element in a Stream, generalized so k is passed to each query (0-indexed in the examples).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def insert(x):
    items.append(x)
def find_largest(k):
    ordered = sorted(items, reverse=True)
    return ordered[k]
```

BRUTE FORCE

TIME	SPACE
O(n log n) per query	O(n)

WHY IT IS SLOW

Every query re-sorts the whole history, O(n log n). A sorted list with binary search makes each query O(1) after an O(log n) search plus shift; for a fixed k a size-k heap is O(log k).

OPTIMAL SOLUTION · Sorted list with bisect (k chosen per query)

TIME	SPACE
O(log n) search + O(n) shift per insert, O(1) query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

FOLLOW-UP 2 What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

FOLLOW-UP 3 Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and k-th).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Insert + Find K-th Largest
#
# Examples: insert(3) insert(3) insert(2) findLargest(0) -
>
# 3 findLargest(1) -> 3 findLargest(2) -> 2
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. What if values can also be deleted?
# 2. What is the trade-off between the two designs?
# 3. Values can also be deleted.
#
# INTERVIEW NOTES
# If k can change from query to query you need every
# value, in a structure that supports rank queries; if k
# is fixed you only need the k largest, and a min-heap of
# size k gives the answer at its root. Ask which of the
# two it is: it decides everything.
#
# BRUTE FORCE (say it first): Store everything and sort
# for every query
# Append values to a list; on a query sort it in
# descending order and read index k. Correct and the
# oracle; each query re-sorts everything. Time O(n log n)
# per query. Space O(n).


import heapq
from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Store everything and sort
# for every query | time O(n log n) per query
class StreamKthBrute:
    def __init__(self):
        self.a = []

    def insert(self, x):
        self.a.append(x)

    def findLargest(self, k):
        s = sorted(self.a, reverse=True)
        return s[k] if 0 <= k < len(s) else None


# BASE QUESTION (optimal): Sorted list with bisect (k
# chosen per query)
```

```
# Idea: Keep all values in an ascending list with insort.
# The k-th largest (0-indexed as in the examples) is a[-1
# - k].
# FOLLOW-UP 1: What if values can also be deleted?
# Answer: The heap can no longer answer because removing
# an arbitrary element is O(k).
# FOLLOW-UP 3: Values can also be deleted.
# Answer: A heap cannot delete an arbitrary value
# efficiently.
class StreamKth:
    """Any k, with deletes: a sorted list."""

    def __init__(self):
        self.a = []

    def insert(self, x):
        # keep the list sorted: O(log n) search, O(n)
        # shift
        insort(self.a, x)

    # FOLLOW-UP 1, 3: values can also be deleted
    def delete(self, x):
        # remove one copy of x if present
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False

    def find_largest(self, k):
        # k = 0 is the largest value
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None


# FOLLOW-UP 2: What is the trade-off between the two
# designs?
# Answer: The heap is the smallest and fastest when k is
# fixed and small (O(log k), O(k) memory) but cannot
# change k or delete. The sorted list answers any k and is
# simplest, at O(n) memory and a linear shift per insert.
class FixedKth:
    """Fixed k, inserts only: a min-heap of the best k."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def insert(self, x):
        heapq.heappush(self.heap, x)
        # keep only the k largest values
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

    def kth_largest(self):
        # the smallest of the k largest
        if len(self.heap) < self.k:
            return None
        return self.heap[0]


if __name__ == "__main__":
    s = StreamKth()
    for x in (3, 3, 2):
        s.insert(x)
    top3 = [s.find_largest(k) for k in range(3)]
    assert top3 == [3, 3, 2]
    assert s.find_largest(3) is None
    # deletes work too
    assert s.delete(3) and not s.delete(9)
    assert [s.find_largest(k) for k in range(2)] == [3, 2]
    f = FixedKth(2)
    assert f.kth_largest() is None
    for x in (5, 1, 9, 7):
        f.insert(x)
    # the two largest are 9 and 7, so the 2nd largest is 7
    assert f.kth_largest() == 7
```

NOTES FROM CANDIDATES

An order-statistics structure (size-augmented BST / balanced tree / skiplist) gives O(log n) for both ops.
An order-statistics Fenwick tree over a coordinate-compressed value range gives O(log V).
A naive sorted array gives O(log n) find but O(n) insert; mention as a baseline.
Two heaps (the canonical streaming-median pattern) only work for a fixed k; the interviewer wants arbitrary k per call.

PREPARATION

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

STUDY PATH: question 22 of 167 (Core 90)

Priority score 3.92 | Binary trees and BSTs | asked 34 days ago

HARD | IN APP LC 1325 | PracHub | Technical Screen | 7 of 7

Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

POSTED 2026-09-16 | 34 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 10.6 30 DAYS 0 PRIORITY 164/228

ABOUT THIS QUESTION

Topic: Core 90 - Binary trees and BSTs

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1325 Delete Leaves With a Given Value

LeetCode match: LC 1325 Delete Leaves With a Given Value

Posted on PracHub: 2026-09-16 (updated 2026-09-20)

Original: https://prachub.com/coding-questions/repeatedly-delete-leaves-matching-a-target-value-from-a-binary-tree

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

APPROACH

Post-order DFS: process children first, then delete the node if it became a leaf with value == target. O(n).

HOW TO THINK ABOUT IT

'Repeatedly' hides an order: a node becomes deletable only after its children are processed. That is a post-order dependency, so a single bottom-up pass does what many passes would do. Whenever a rule says 'repeat until nothing changes' on a tree, ask whether children-first order collapses it to one pass.

Basis: LeetCode 1325, Delete Leaves With a Given Value.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def delete_pass(node):
    if node is None:
        return None, 0
    node.left, a = delete_pass(node.left)
    node.right, b = delete_pass(node.right)
    if is_leaf(node) and node.val == target:
        return None, a + b + 1
    return node, a + b
while True:
    root, removed = delete_pass(root)
    if removed == 0:
        break
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A chain of matching nodes needs one pass per node, n passes of O(n). Post-order (children first) deletes the whole chain in a single O(n) pass.

OPTIMAL SOLUTION · One post-order pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is a 100,000-node chain. How do you avoid a stack overflow?

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

FOLLOW-UP 2 Delete any node whose whole subtree sums to zero, or whose value matches a predicate.

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. **Cost:** one post-order pass, O(n) time and O(h) space.

FOLLOW-UP 3 Return how many nodes were deleted.

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. **Cost:** O(1) extra, counted inside the same O(n) pass.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Repeatedly Delete Leaves Matching a
# Target Value From a Binary Tree
#
# Given a binary tree and an integer target, delete every
# leaf whose value equals target. Deleting a leaf can turn
# its parent into a leaf; if that parent's value also
# equals target, it must be deleted as well, and so on,
# until no leaf with value target remains. Return the
# resulting tree, which is empty if the root itself ends
# up deleted. ### Function Signature
# remove_target_leaves(tree: list[int | None], target:
# int) -> list[int | None] ### Tree Encoding The input and
# the output use the same level-order encoding: - An empty
# tree is []. - Otherwise the first element is the root's
# value. Present nodes are then visited in breadth-first
# order, left to right, and for each one its left child
# and then its right child are appended, using None for an
# absent child. Children of absent nodes are not listed. -
# Trailing None values are omitted.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is a 100,000-node chain. How do you avoid a
# stack overflow?
# 2. Delete any node whose whole subtree sums to zero, or
# whose value matches a predicate.
# 3. Return how many nodes were deleted.
#
# INTERVIEW NOTES
# 'Repeatedly' hides an order: a node becomes deletable
# only after its children are processed. That is a post-
# order dependency, so a single bottom-up pass does what
# many passes would do. Whenever a rule says 'repeat until
# nothing changes' on a tree, ask whether children-first
# order collapses it to one pass.
#
# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing
# Each pass deletes only the current leaves with the
# target value; a parent that becomes a matching leaf
# waits for the next pass. On a chain of n matching nodes
# that is n passes of O(n) each. Time O(n * h). Space
# O(h).


from collections import deque


# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing | time O(n * h)
def remove_leaves_brute(root, target):
    def one_pass(node):
        if node is None:
            return None, False
        if node.left is None and node.right is None:
            return (
                (None, True)
                if node.val == target
                else (node, False)
            )
        node.left, a = one_pass(node.left)
        node.right, b = one_pass(node.right)
        return node, a or b

    changed = True
    while changed:
        root, changed = one_pass(root)
    return root


# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent.
def remove_leaves(root, target):
    # recursive post-order: children first, then the node
    deleted = 0

    def dfs(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        # a leaf with the target value disappears
        if not node.left and not node.right \
                and node.val == target:
            deleted += 1
            return None
        return node

    return dfs(root), deleted


# FOLLOW-UP 1: The tree is a 100,000-node chain. How do
# you avoid a stack overflow?
# Answer: Collect nodes in pre-order with an explicit
# stack and remember each node's parent, then walk that
# list in reverse (children before parents) and unlink
```

```
# matching leaves. This keeps the post-order guarantee
# without recursion.
def remove_leaves_iterative(root, target):
    # no recursion: safe for a 100,000-node chain
    if root is None:
        return None, 0
    parent, order, stack = {root: None}, [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[child] = node
                stack.append(child)
    deleted = 0
    # reversed order: children are handled before parents
    for node in reversed(order):
        if node.left or node.right or node.val != target:
            continue
        deleted += 1
        up = parent[node]
        if up is None:
            return None, deleted
        if up.left is node:
            up.left = None
        else:
            up.right = None
    return root, deleted


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build(values):
```

```
    # level-order list with None for missing children
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        # next two values are this node's children
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


def to_list(root):
    # the same encoding, with trailing Nones removed
    out, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        out.append(node.val if node else None)
        if node:
            queue.append(node.left)
            queue.append(node.right)
    while out and out[-1] is None:
        out.pop()
    return out


def remove_target_leaves(values, target):
    root, _ = remove_leaves(build(values), target)
    return to_list(root)
```

```
if __name__ == "__main__":
    tree = [1, 2, 3, 2, None, 2, 4]
    result = remove_target_leaves(tree, 2)
    assert result == [1, None, 3, None, 4]
    # the whole tree can disappear
    assert remove_target_leaves([1, 1, 1], 1) == []
    assert remove_target_leaves([], 5) == []
    root, count = remove_leaves(build(tree), 2)
    assert count == 3
```

```
    again = remove_leaves_iterative(build(tree), 2)
    assert again[1] == 3
    assert to_list(again[0]) == to_list(root)
    # a 100,000 node chain works without recursion
    chain = TreeNode(7)
    cur = chain
    for _ in range(99999):
        cur.left = TreeNode(7)
        cur = cur.left
    gone = remove_leaves_iterative(chain, 7)
    assert gone == (None, 100000)
```

FUNCTION AND TEST CASES

Signature

```
def remove_target_leaves(tree: list[int | None], target: i
nt) -> list[int | None]:
```

Sample call

```
remove_target_leaves([5, 7, 7, 7, 3, None, 7], 7)
```

Test cases

Input	Expected
`([5, 7, 7, 7, 3, None, 7], 7)`	`[5, 7, None, None, 3]`
`([3, 3, 1], 3)`	`[3, None, 1]`